CS2310: OPERATING SYSTEMS

Group	18
Class	SY Computer Engineering A
Batch	3
PRN	12411297

PHASE 2: A MULTIPROGRAMMING OPERATING SYSTEM

PROBLEM STATEMENT

To implement phase 1 of an operating system, including the following:

1. CPU/Machine simulation
2. Supervisor call through interrupt
3. Paging
4. Error Handling
5. Interrupt Generation and Servicing
6. Process Data Structure

ASSUMPTIONS MADE

While implementing phase 1, we had made the following assumptions:

1. Jobs entered without error in input file
2. No physical separation between jobs
3. Job outputs separated in output file by 2 blank lines
4. Program loaded in memory starting at location 00
5. No multiprogramming, load and run one program at a time
6. SI interrupt for service request

In phase 2, there is a slight change in assumptions, with some additions:

1. Jobs may now have program errors (update)
2. PI interrupt for program errors introduced
3. Paging introduced, page table stored in real memory
4. Program pages allocated one of 30 memory blocks using a random number generator. Load and run one program at a time (update)
5. Time limit, line limit, out-of-data errors introduced
6. TI interrupt for time-out error introduced

OVERVIEW

Phase 2 of implementation introduces 3 new concepts, namely, virtual memory, paging and error handling. The virtual machine now has an upgraded memory space including 300 words of physical memory and 100 words for virtual memory. All registers and instructions remain the same, and all the addresses given in the instructions refer to the virtual memory locations. These will later be converted to a physical address via an address translation mechanism.

In computers, physical memory is limited by the size of the address bus. If the address bus is of n-bits, then the physical memory will have 2^n memory locations. To store more memory than available, we can "trick" the computer into believing that there's extra space in the form of a virtual memory. This allows us to enter parts of the virtual memory into the physical memory and swap out a portion to create more space as and when required. This can be done with the concept of paging.

a. Paging

Paging is a contiguous memory allocation scheme in which the main memory gets divided into fixed size blocks known as **"frames"** and the virtual memory is divided into **"pages"** of the same size as frames. When we need to execute a page of the program, it can be brought into the main memory, where it is randomly allocated a frame for its storage.

A **page table** is a data structure that keeps track of the frame numbers to which pages have been allocated. Whenever the OS detects that a new process/job has arrived, it creates a new page table for it. The page table itself resides in the main memory and is randomly allocated to a frame. A **page table register** stores the starting address of the block where the page table is located.

However, as stated previously, the addresses written in the instructions refer to the virtual memory. In order to store data and instructions in the main memory, we will need to translate the addresses using an address translation mechanism.

b. Address Translation

$$PTE = PTR + VA/10$$
$$PA = M[PTE]*10 + VA\%10$$

Where PTE is the page table entry
PTR is the page table register
VA is the virtual address
PA is the physical address
M[PTE] is the memory location of the PTE

Thus, to get the physical address of a virtual address, we first need to find the entry in the page table that will redirect us to the correct frame for that page. Then, we can access each individual memory location in that particular frame.

For example, let us try to bring the following page in the main memory:

00	GD10
01	PD10
02	H
…	
09	

Assuming that our page table register is in the 7th frame. Thus, PTR = 70, for our system. For the first instruction:

VA = 00

PTE = 70 + 0/10 = 70

Now, we will randomly allocate a frame to the page coming in. Let the allocated frame be 10. Store this in the page table:

70	0010
71	
…	
79	

Now, M[PTE] stores the allocated frame number. To get the PA,

PA = 10*10 + 0%10 = 100

So, in frame 10, we enter the instruction:

100	GD10
101	
…	
109	

Repeating this process, the frame would finally look like:

100	GD10
101	PD10
102	H
…	
109	

When there is no entry in the page table for a PTE calculated during address translation, a **page fault** occurs. This means that there is a part of a program in the virtual memory that is not physically present in our system.

c. Error Handling

7 types of errors were introduced. These are as follows:

1. Opcode error - When the entered opcode differs from the 7 recognised instructions (GD, PD, LR, SR, CR, BT, H), the system fails to recognise the instruction. This is called an opcode error.

2. Operand error - When the entered operand (i.e virtual address) for an instruction is either out of bounds or not numerical, it is an operand error.

3. Time limit exceeded - In the $AMJ control card, the 4 digits that follow the job ID denote the time required to execute the job. In phase 2, GD and SR instructions require 2 time periods to execute (1 to handle valid page faults and the other to execute the instruction itself) while the rest of the instructions require 1 time period to execute. If the program runs for longer than it should, then a time limit error will occur.

4. Line limit exceeded - In the $AMJ control card, the last 4 digits denote the number of lines of output that will be written by a job. If the program, while executing, tries to write more lines than allotted, a line limit error will occur.

5. Out of data error - When the program expects to receive input from a data card that does not exist in our job, an out of data error occurs.

6. Valid page fault - When GD and SR try to insert into the memory, the corresponding PTE is calculated, however, since there was no program loaded into the desired location before these instructions were called, a page fault occurs. This page fault is said to be

valid, since it shows that the system is working correctly, and to resolve it, we allocate a frame to the part of the program being added to the memory.

7. Invalid page fault - When an instruction other than GD or SR leads to a page fault, it is said to be an invalid page fault, since the system expects some program to have been loaded at a desired location, and it was not found.

In order to flag some of these errors, we make use of interrupts. In phase 1, we had implemented system interrupts for GD, PD and H (SI = 1, 2 and 3 respectively). Additionally, in phase 2, we have 2 more interrupts:

- Programming Interrupt (PI) = 1 → Opcode error
  2 → Operand error
  3 → Page fault
- Timing interrupt (TI) = 2 → Time limit exceeded

Along with SI, the above 2 form certain combinations that lead to program termination, which will be discussed in the next section.

d. Process Control Block

A process control block (or PCB) is a data structure that stores identifying information about a process. In our implementation, the PCB includes:

- Process ID (pid)
- Total time limit (TTL)
- Total line limit (TLL)
- Total time counter (TTC)
- Total line counter (TLC)

A PCB is created for every new job that comes into the system. The counters for time and line limit help us recognise when a TLE or LLE occurs.

FUNCTION DESCRIPTION

1. main() - This is the entry point to the program. Its only purpose is to call the load() function to begin the program.

2. allocate() - This function is used to randomly allocate a frame number (0-29) whenever it is called. It does this by iterating through a shuffled array containing integers 0-29.

3. load() - This function is used to start reading the input file. It reads the file line by line and checks for the following:
    a. If the line starts with $AMJ - A new program has begun.Extract the process ID, TTL and TLL from the $AMJ card and update the values in the PCB. Call init() to initialize the memory and registers.
    b. If the line starts with $DTA - Programs have been loaded, and the data follows. Call executeprog() to start the program execution.
    c. If the line starts with $END - The program is over. Call displaymem() to display the memory contents.
    d. If the line starts with anything else, it must surely be the program instructions. Load the instructions in the memory. Since every new program card will be loaded into a new frame, we will call the allocate() function to allocate a frame to the page and update the page table in the memory. Then start filling the contents into that frame.

4. init() - This function clears all the values in the memory and registers and replaces them with our chosen empty byte character, '-' for the next program. We also allocate a frame for the page table to reside in and update the page table register accordingly.

5. addresstranslate() - This function implements the address translation mechanism that was defined previously. If a page fault is detected, it sets PI to 3 and calls master mode to handle it. However, before any of this, if the entered virtual address cannot properly be parsed to an integer, it will throw an operand error and set PI = 2.

6. executeprogram() - This function reads the opcode of the instructions stored and executes them accordingly. It keeps a track of the number of time units the program has been running for and increments the counter accordingly. It uses a switch block to match the opcode and depending on the opcode, it does the following:
    a. GD, PD, H: Since these instructions trigger the system interrupt, it sets the SI bit to 1, 2, 3 respectively and calls the mastermode() function to switch to master mode to handle the interrupt.
    b. LR, SR, CR: It parses the address given in the command, performs address translation, and accordingly sets the value of the register or memory location or compares values and sets C.

c. BT: It checks if the toggle bit is set or unset, and accordingly increments the IC (condition did not hold) or updates it to the address to branch to (condition held true).
d. If the entered opcode does not match any of the above, an opcode error has occurred, set PI = 1, and go to master mode..

7. mastermode(Scanner sc) - Depending on the value of SI, PI and TI, it will either handle the error and continue or terminate the program:
    a. Calls read() if SI=1 & TI=0.
    b. Calls write() if SI=2 & TI=0.
    c. Calls terminate() if SI=3 & TI=0, with message **Process exited with status 0.**
    d. Calls terminate() if SI=1 & TI=2, with error message **Time Limit Exceeded.**
    e. Calls write() first, then terminate() if SI=2 & TI=2, with error message **Time Limit Exceeded.**
    f. Calls terminate() if SI=3 & TI=2, with message **Process exited with status 0.**
    g. Calls terminate() if PI=1 & TI=0, with message **Opcode Error.**
    h. Calls terminate() if PI=2 & TI=0, with message **Operand Error.**
    i. Calls terminate() if PI=3 & TI=0, with message **Invalid Page Fault**, if the page fault is not caused by GD or SR.
    j. Calls terminate() if PI=1 & TI=2, with message **Time Limit Exceeded, Opcode Error.**
    k. Calls terminate() if PI=2 & TI=2, with message **Time Limit Exceeded, Operand Error.**
    l. Calls terminate() if PI=3 & TI=2, with message **Time Limit Exceeded, Invalid Page Fault.**
    m. Handles Valid Page Faults. Checks whether the page fault is caused by GD or SR, if it is caused by either of those, allocate() is called to get a random frame number

8. read(Scanner sc) - It moves to the next line of the file and reads the data from the data card into the memory at the specified location. If the data read begins with $END, calls terminate() with error message **Out of Data Error.**

9. write() - Increments Line Limit Counter. Checks if Line Limit Counter is greater than Total Line Limit, if yes, calls terminate() with message **Line Limit Exceeded**. Else, it writes the output to the output file.

10. displaymem() - It displays the contents of the memory.

11. terminate() - It adds two new lines to the output file to separate the outputs of two jobs.

INPUT FILES

With Errors

```
┌──(muggloaf㉿alesha)-[~/phase2]
└─$ cat inputE.txt
$AMJ000100040001
GP10PP10H
$DTA
Hello World!
$END0001
$AMJ000200150001
GD2AGD30GD40GD5OLR20CR30BT09PD50HPD40
H
$DTA
VIT
VIIT
SAME
NOT SAME
$END0002
$AMJ000300320001
GD20LR20SR30SR31SR32SR33SR40SR43SR50SR53
SR60SR61SR62SR63PD30PD40PD50PD60H
$DTA
*
$END0003
$AMJ000400140001
GD30GD40GD50LR30CR50BT12CR51BT16CR52BT20
CR53BT24LR40SR51PD50HLR40SR52PD50H
LR40SR53PD50HLR40SR54PD50H
$DTA
car
cat
The car purrs
$END0004
$AMJ000500330001
GD50GD60LR50CR60BT11CR61BT19CR62BT27CR63
BT35LR61SR70LR62SR71LR63SR72PD70HLR60
SR70LR62SR71LR63SR72PD70HLR60SR70LR61
SR71LR63SR72PD70HLR60SR70LR61SR71LR62
SR72PD70H
$DTA
The
$END0005
$AMJ000600220008
GD20LR20SR30SR31SR40SR41SR42PD20PD30PD40
PD30PD20PD20PD20PD20H
$DTA
*
$END0006
$AMJ000700160001
GD20LR20SR33LR21SR32LR22SR31LR23SR30PD80
H
$DTA
HOW ARE YOU NOW
$END0007
```

Without Errors

```
┌──(muggloaf㉿alesha)-[~/phase2]
└─$ cat inputNE.txt
$AMJ000100040001
GD10PD10H
$DTA
Hello World!
$END0001
$AMJ000200150001
GD20GD30GD40GD50LR20CR30BT09PD50HPD40
H
$DTA
VIT
VIIT
SAME
NOT SAME
$END0002
$AMJ000300320004
GD20LR20SR30SR31SR32SR33SR40SR43SR50SR53
SR60SR61SR62SR63PD30PD40PD50PD60H
$DTA
*
$END0003
$AMJ000400350001
GD30GD40GD50LR30CR50BT12CR51BT16CR52BT20
CR53BT24LR40SR51PD50HLR40SR52PD50H
LR40SR53PD50HLR40SR54PD50H
$DTA
car
cat
The car purrs
$END0004
$AMJ000500330001
GD50GD60LR50CR60BT11CR61BT19CR62BT27CR63
BT35LR61SR70LR62SR71LR63SR72PD70HLR60
SR70LR62SR71LR63SR72PD70HLR60SR70LR61
SR71LR63SR72PD70HLR60SR70LR61SR71LR62
SR72PD70H
$DTA
The
The apple is red
$END0005
$AMJ000600220008
GD20LR20SR30SR31SR40SR41SR42PD20PD30PD40
PD30PD20PD20PD20PD20H
$DTA
*
$END0006
$AMJ000700160001
GD20LR20SR33LR21SR32LR22SR31LR23SR30PD30
H
$DTA
HOW ARE YOU NOW
$END0007
```

OUTPUT FILES

With Errors

```
┌──(muggloaf㉿alesha)-[~/phase2]
└─$ cat output.txt
Opcode Error
SI : 0  PI : 1  TI : 0   IR : GP10   IC : 0


Operand Error
SI : 0  PI : 2  TI : 0   IR : GD2A   IC : 1


*    *    *    *
Line Limit Exceeded
SI : 0  PI : 0  TI : 0   IR : PD40   IC : 16


The cat  purrs
Time Limit Exceeded
SI : 2  PI : 0  TI : 2   IR : PD50   IC : 15


Out of data
SI : 0  PI : 0  TI : 0   IR : GD60   IC : 2


*
*    *
*    *    *
*    *
*
*
*
*
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 16


Invalid Page Fault
SI : 0  PI : 3  TI : 0   IR : PD80   IC : 10
```

Without Errors

```
┌──(muggloaf㉿alesha)-[~/phase2]
└─$ cat outputNE.txt
Hello World!
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 3


NOT SAME
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 9


*    *    *    *
*              *
*              *
*    *    *    *
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 19


The cat  purrs
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 16


apple is red
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 19


*
*    *
*    *    *
*    *
*
*
*
*
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 16


NOW  YOU ARE HOW
Process exited with status 0
SI : 3  PI : 0  TI : 0   IR : H---   IC : 11
```

SPOTLIGHT ON EACH JOB

1. *Print "Hello World!"*

Type of error introduced (first occurrence highlighted): Opcode error

Job without Errors	Job with Errors
Input	
$AMJ000100040001 **GD10PD10H** **$DTA** **Hello World!** **$END0001**	**$AMJ000100040001** **GP10PP10H** **$DTA** **Hello World!** **$END0001**
Output	
Hello World! Process exited with status 0 SI : 3 PI : 0 TI : 0 IR : H--- IC : 3	Opcode Error SI : 0 PI : 1 TI : 0 IR : GP10 IC : 0

2. *Check if two strings are the same (check if VIT and VIIT are the same)*

Type of error introduced (first occurrence highlighted): Operand error

Job without Errors	Job with Errors
Input	
$AMJ000200150001 **GD20GD30GD40GD50LR20CR30BT09PD50HPD40H** **$DTA** **VIT** **VIIT** **SAME** **NOT SAME** **$END0002**	**$AMJ000200150001** **GD20GD30GD40GD5OLR20CR30BT09PD50HPD40H** **$DTA** **VIT** **VIIT** **SAME** **NOT SAME** **$END0002**
Output	
NOT SAME Process exited with status 0 SI : 3 PI : 0 TI : 0 IR : H--- IC : 9	Operand Error SI : 0 PI : 2 TI : 0 IR : GD5O IC : 4

3. *Print a rectangle pattern with '*'*

Type of error introduced (first occurrence highlighted): Line limit exceeded

Job without Errors	Job with Errors
Input	
$AMJ000300320004 **GD20LR20SR30SR31SR32SR33SR40SR43SR50SR53** **SR60SR61SR62SR63PD30PD40PD50PD60H** **$DTA** ***** **$END0003**	**$AMJ00030032**0001 **GD20LR20SR30SR31SR32SR33SR40SR43SR50SR53** **SR60SR61SR62SR63PD30PD40PD50PD60H** **$DTA** ***** **$END0003**
Output	

`*  *  *  *`
`*  *`
`*  *`
`*  *  *  *`
`Process exited with status 0`
`SI : 3  PI : 0  TI : 0  IR : H---  IC : 19`
	`*  *  *  *`
	`Line Limit Exceeded`
	`SI : 0  PI : 0  TI : 0  IR : PD40  IC : 16`

4. *Replace one string with another (Replace "car" with "cat")*

Type of error introduced (first occurrence highlighted): Time limit exceeded

Job without Errors	Job with Errors
Input	
$AMJ000400350001 **GD30GD40GD50LR30CR50BT12CR51BT16CR52BT20** **CR53BT24LR40SR51PD50HLR40SR52PD50H** **LR40SR53PD50HLR40SR54PD50H** **$DTA** **car** **cat** **The car purrs** **$END0004**	**$AMJ0004**0002**0001** **GD30GD40GD50LR30CR50BT12CR51BT16CR52BT20** **CR53BT24LR40SR51PD50HLR40SR52PD50H** **LR40SR53PD50HLR40SR54PD50H** **$DTA** **car** **cat** **The car purrs** **$END0004**
Output	

`The cat  purrs`
`Process exited with status 0`
`SI : 3  PI : 0  TI : 0  IR : H---  IC : 16`
	`Time Limit Exceeded`
	`SI : 1  PI : 0  TI : 2  IR : GD40  IC : 2`

5. *Remove the article “The” from a sentence*

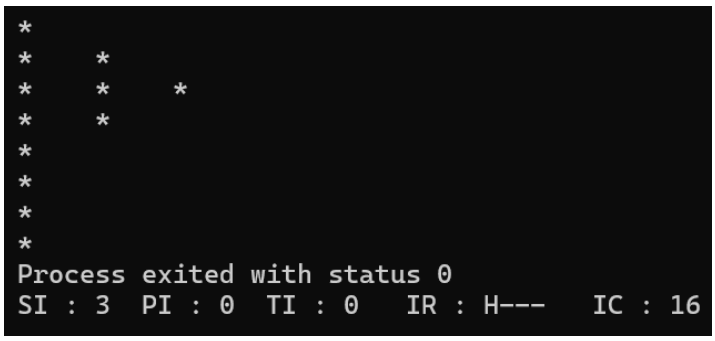
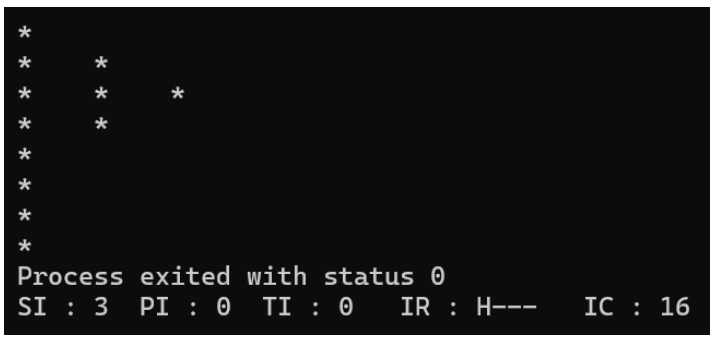
Type of error introduced (first occurrence highlighted): Out of data

Job without Errors	Job with Errors
Input	
$AMJ000500330001 GD50GD60LR50CR60BT11CR61BT19CR62BT27CR63 BT35LR61SR70LR62SR71LR63SR72PD70HLR60 SR70LR62SR71LR63SR72PD70HLR60SR70LR61 SR71LR63SR72PD70HLR60SR70LR61SR71LR62 SR72PD70H $DTA The The apple is red $END0005	$AMJ000500330001 GD50GD60LR50CR60BT11CR61BT19CR62BT27CR63 BT35LR61SR70LR62SR71LR63SR72PD70HLR60 SR70LR62SR71LR63SR72PD70HLR60SR70LR61 SR71LR63SR72PD70HLR60SR70LR61SR71LR62 SR72PD70H $DTA The $END0005
Output	
apple is red Process exited with status 0 SI : 3 PI : 0 TI : 0 IR : H--- IC : 19	Out of data SI : 0 PI : 0 TI : 0 IR : GD60 IC : 2

6. *Print flag pattern with ‘*’*

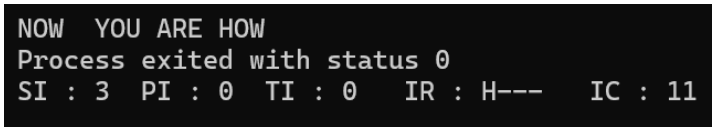
Type of error introduced (first occurrence highlighted): Valid Page Fault

Job without Errors	Job with Errors
Input	
$AMJ000600220008 GD20LR20SR30SR31SR40SR41SR42PD20PD30PD40 PD30PD20PD20PD20PD20H $DTA * $END0006	$AMJ000600220008 GD20LR20SR30SR31SR40SR41SR42PD20PD30PD40 PD30PD20PD20PD20PD20H $DTA * $END0006
Output	

7. *Print sentence in reverse*

Type of error introduced (first occurrence highlighted): Invalid Page Fault

Job without Errors	Job with Errors
Input	
$AMJ000700160001 **GD20LR20SR33LR21SR32LR22SR31LR23SR30PD30** **H** **$DTA** **HOW ARE YOU NOW** **$END0007**	**$AMJ000700160001** **GD20LR20SR33LR21SR32LR22SR31LR23SR30PD80** **H** **$DTA** **HOW ARE YOU NOW** **$END0007**
Output	
	

CODE

```
import java.util.*;
import java.io.*;

public class phase2{
    //4 registers in cpu
    public static char[] R = new char[4];
    public static char[] IR = new char[4];
    public static int IC;
    public static boolean C;
    public static int SI;
    public static int TI;
    public static int PI;

    //memory
    public static char[][] mem = new char[300][4];

    public static char[] buffer = new char[40];
    public static char[] PTR=new char[4];

    public static int numinstr; //no. of instrs in prog card
    public static int TLC;
    public static int TTL;
    public static int LLC;
    public static int TLL;
    public static int gdcall;
    public static int srcall;
    public static int framevpf; //frame assigned during valid pg fault

    public static ArrayList<Integer> framenums=new
ArrayList<Integer>();
    public static int frameidx;
    public static int pagenum;
    public static int pgtableloc; //row in page table where we need to
make an entry
     public static int pagetableAT; //pte after valid pg fault is
handled
    public static int LLE; //line limit exceeded
    public static int OOD; //out of data error

    static PCB pro = new PCB();
    public static Scanner sc;

    public static void main(String[] args) throws Exception{
```

```
        load();
    }

    public static void load() throws Exception{
        String line, word;
        int lineptr = 0;
        int instcnt = 0;
        pagenum=49;
        int frametofill;
        File f = new File("inputE.txt");
        sc = new Scanner(f);

        while(sc.hasNextLine()){
            line = sc.nextLine();
            if(line.length()>4) word = line.substring(0,4);
            else word = line;

            if(word.equals("$AMJ")){
                lineptr = 0; instcnt = 0;
                numinstr = Integer.parseInt(line.substring(8,12));
                pro.pid=Integer.parseInt(line.substring(4,8));
                TTL=Integer.parseInt(line.substring(8,12));
                pro.TTL=TTL;
                TLL=Integer.parseInt(line.substring(12));
                pro.TLL=TLL;
                init();
            }
               else if(word.equals("$DTA")) {pro.currentstat="EXECUTE";
executeprog();}
              else if(word.equals("$END")) {pro.currentstat="FINISHED";
displaymem(); continue;}
            else{
                pro.currentstat="FETCH";
                instcnt = 0;
                line = line.trim();
                char[] instructions = line.toCharArray();

                 for(int i = 0; i<instructions.length; i++) buffer[i] =
instructions[i];

                frametofill = allocate();
                String idk = Integer.toString(frametofill);
                if(frametofill<10) idk = "0"+idk;
                char[] frameAsChar = idk.toCharArray();
                mem[pgtableloc][0] = 'P';
```

```
            mem[pgtableloc][1] = (char)pagenum;
            pagenum++;
            mem[pgtableloc][2] = frameAsChar[0];
            mem[pgtableloc][3] = frameAsChar[1];
            pgtableloc++;
            lineptr = frametofill*10;

            for(int i = 0; i<instructions.length; i++){
                if(instcnt!=0 && instcnt%4==0){
                    lineptr++;
                    numinstr++;
                }
                if(instructions[i]=='H'){
                    mem[lineptr][instcnt%4]=buffer[i];
                    numinstr++;
                    instcnt=((instcnt/4)+1)*4;
                }
                else{
                    mem[lineptr][instcnt%4]=buffer[i];
                    instcnt++;
                    if(i==(instructions.length-1)) numinstr++;
                }
            }

            for(int i = 0; i<40; i++) buffer[i] = '-';
        }
    }
}

public static int allocate(){
    int n=framenums.get(frameidx);
    frameidx++;
    return n;
}

public static void init() throws Exception{
    IC = 0;
    C = false;
    SI = 0; TI=0; PI=0;
    TLC=0; LLC=0;
    pagenum=49;
    numinstr = 0;
    LLE=0; OOD=0;
    gdcall=0; srcall=0;
    framevpf=0; frameidx=0;
```

```
        R[0] = 0; R[1] = 0; R[2] = 0; R[3] = 0;
        IR[0] = 0; IR[1] = 0; IR[2] = 0; IR[3] = 0;

        framenums.clear();
        for(int i=0; i<30; i++) framenums.add(i);
        Collections.shuffle(framenums);

        int temp=allocate();
        pgtableloc=temp*10;
        String frame=Integer.toString(temp);
        if(temp<10) frame="00"+frame+"0";
        else frame="0"+frame+"0";
        char[] r=frame.toCharArray();

        PTR[0]=r[0];
        PTR[1]=r[1];
        PTR[2]=r[2];
        PTR[3]=r[3];
        System.out.println("PTR : "+PTR[0]+PTR[1]+PTR[2]+PTR[3]);

         for(int i = 0; i<300; i++) {mem[i][0] = '-'; mem[i][1] = '-';
mem[i][2] = '-'; mem[i][3] = '-';}

        for(int i = 0; i<40; i++) buffer[i] = '-';

        File of=new File("output.txt");
        if(of.createNewFile())
        System.out.println("Created an output file : output.txt");
    }

     public static int addresstranslate(String va, Scanner sc) throws
Exception{
        int VA = 0;
        try{
            VA=Integer.parseInt(va);
                 System.out.println("Virtual address: "+VA+" Thingy we
passed: "+va);
        }
        catch(Exception e){
            PI=2;
            mastermode(sc);
        }

        String pageTable=""+PTR[0]+PTR[1]+PTR[2]+PTR[3];
```

```
        int ptrval=Integer.parseInt(pageTable);
        int pte=ptrval+(VA/10);
        pagetableAT=pte;

        if(mem[pte][0]!='P'){
            PI=3;
            mastermode(sc);
            if(PI!=3){
                int RA=framevpf*10;
                RA=RA+VA%10;
                return RA;
            }
            else return -1;
        }
        else{
            String temp=""+mem[pte][2]+mem[pte][3];
            int framenum=Integer.parseInt(temp);
            int RA=(framenum*10)+(VA%10);
            srcall = 0;
            return RA;
        }
    }

    public static void executeprog(){
        int addr, RA;;
        String temp;
        try {
            while(IC!=numinstr){
                pro.completiontime = TLC;
                System.out.println("Value of TLC: "+TLC);
                if(TLC>=TTL) TI = 2;
                System.out.println("\n\nValue of IC : "+IC);

                temp = "";
                addr = 0;

                String temp2 = "" + IC;
                RA = addresstranslate(temp2,sc);
                  System.out.println("Real Address corresponding to IC
"+IC+" is : "+RA);

                for(int i = 0; i<4; i++) IR[i] = mem[RA][i];

                String opcode = "" + IR[0] + IR[1];
                if (IR[0] == 'H') {
```

```
        System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
        IC++; SI=3;
        mastermode(sc);
        TLC++;
        break;
    }

    switch(opcode){
        case "GD":
           System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
            IC++; SI=1;
            mastermode(sc);
            TLC++;
            break;

        case "PD":
           System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
            IC++; SI=2;
            mastermode(sc);
            TLC++;
            break;

        case "LR":
           System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
            IC++;
            temp = "" + IR[2] + IR[3];
            addr = Integer.parseInt(temp);
            addr = addresstranslate(temp,sc);
            for(int i = 0; i<4; i++) R[i] = mem[addr][i];
            TLC++;
            break;

        case "SR":
           System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
            IC++;
            temp = "" + IR[2] + IR[3];
            addr = Integer.parseInt(temp);
            srcall = 1;
            addr = addresstranslate(temp,sc);
            for(int i = 0; i<4; i++) mem[addr][i] = R[i];
            TLC++;
            break;

        case "CR":
           System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
```

```
                    IC++;
                    temp = "" + IR[2] + IR[3];
                    addr = Integer.parseInt(temp);
                    addr = addresstranslate(temp,sc);
                    for(int i = 0; i<4; i++){
                        if(R[i] == mem[addr][i]) C = true;
                        else {C = false; break;}
                    }
                    TLC++;
                    break;

                case "BT":
                   System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
                    if(C==true){
                        temp = "" + IR[2] + IR[3];
                        addr = Integer.parseInt(temp);
                        IC = addr;
                    }
                    else IC++;
                    TLC++;
                    break;

                default: {
                   System.out.println(""+IR[0]+IR[1]+IR[2]+IR[3]);
                    System.out.println("No opcode matches");
                    PI = 1;
                    mastermode(sc);
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void mastermode(Scanner sc) throws Exception{
    System.out.println("Entered masterMode()");
    System.out.println("SI : "+SI);
    System.out.println("TI : "+TI);
    System.out.println("PI : "+PI);

    if(TLC>=TTL) TI=2;
    if(TI==0 && SI==1){SI=0; read(sc);}
    else if(TI==0 && SI==2){SI=0; write(sc);}
```

```
         else if(TI==0 && SI==3) terminate("Process exited with status
0",sc);
        else if(TI==2 && SI==1) terminate("Time Limit Exceeded",sc);
            else if(TI==2 && SI==2){write(sc); terminate("Time Limit
Exceeded",sc);}
         else if(TI==2 && SI==3) terminate("Process exited with status
0",sc);

        else if(TI==0 && PI==1) terminate("Opcode Error",sc);
        else if(TI==0 && PI==2) terminate("Operand Error",sc);
        else if(TI==0 && PI==3){
            if(gdcall==1){
                gdcall=0;
                int frametofill=allocate();
                     System.out.println("Frame assigned while handling
valid page fault caused by GD : "+frametofill);
                String idk=Integer.toString(frametofill);
                if(frametofill<10)
                        idk="0"+idk;
                char[] frameAsChar=idk.toCharArray();
                mem[pagetableAT][0]='P';
                mem[pagetableAT][1]=(char)pagenum;
                pagenum++;
                mem[pagetableAT][2]=frameAsChar[0];
                mem[pagetableAT][3]=frameAsChar[1];
                framevpf=frametofill;
                TLC++;
                PI=0;
            }
            else if(srcall==1){
                srcall=0;
                System.out.println("SR was reset");
                int frametofill=allocate();
                     System.out.println("Frame assigned while handling
valid page fault caused by SR : "+frametofill);
                String idk=Integer.toString(frametofill);
                if(frametofill<10)
                        idk="0"+idk;
                char[] frameAsChar=idk.toCharArray();
                mem[pagetableAT][0]='P';
                mem[pagetableAT][1]=(char)pagenum;
                pagenum++;
                mem[pagetableAT][2]=frameAsChar[0];
                mem[pagetableAT][3]=frameAsChar[1];
                framevpf=frametofill;
```

```
                TLC++;
                PI=0;
            }
            else terminate("Invalid Page Fault",sc);
        }

         else if(TI==2 && PI==1) terminate("Time Limit Exceeded\nOpcode
Error",sc);
                    else  if(TI==2  &&  PI==2)  terminate("Time  Limit
Exceeded\nOperand Error",sc);
                    else  if(TI==2  &&  PI==3)  terminate("Time  Limit
Exceeded\nInvalid Page Fault",sc);
    }

    public static void read(Scanner sc) throws Exception{
        String data = sc.nextLine();
        data=data.trim();
        System.out.println("Data read : "+data);
        String subdata;

        if(data.length()>4) subdata=data.substring(0,4);
        else subdata=data;

        if(subdata.equals("$END")){
            OOD=1;
            terminate("Out of data", sc);
        }
        else{
            char[] datachar = data.toCharArray();
            int length = datachar.length;
            String temp2 = ""+IR[2]+IR[3];
            gdcall = 1;
            int block = addresstranslate(temp2, sc);
            int arraycnt = 0;
            int memcnt = 0;
            while(arraycnt!=length){
                if(memcnt%4==0 && memcnt!=0) block++;
                mem[block][memcnt%4]=datachar[arraycnt];
                arraycnt++;
                memcnt++;
            }
        }
    }

    public static void write(Scanner sc) throws Exception{
```

```
        LLC++;
        pro.linesprinted=LLC;
        if(LLC>TLL){
            LLE=1;
            terminate("Line Limit Exceeded",sc);
        }
        else{
            String temp=""+IR[2]+IR[3];
            int block=addresstranslate(temp,sc);
            if(block!=-1 && OOD!=1){
                  System.out.println("Writing from block : "+block+" to
output file");
                int i,j;
                String data="";
                for(i=block; i<block+10; i++){
                    for(j=0; j<4; j++){
                        if(mem[i][j]!='-') data += mem[i][j];
                        else data += ' ';
                    }
                    if(mem[i][j-1]=='-') data += ' ';
                }
                FileWriter fw = new FileWriter("output.txt",true);
                data = data+"\n";
                fw.write(data);
                fw.close();
            }
        }
    }

    public static void displaymem(){
                for(int i=0; i<300; i++){
                        if(i%10==0 && i!=0)

System.out.println("-----------------------------------------------------
---------------");
                                                    System.out.println(i+"
"+mem[i][0]+mem[i][1]+mem[i][2]+mem[i][3]);
                }
                System.out.println();
                System.out.println();
                System.out.println("===========PCB==========");
        System.out.println("Process ID : "+pro.pid);
        System.out.println("Total Time Limit : "+pro.TTL);
        System.out.println("Total Line Limit : "+pro.TLL);
        System.out.println("Time Given : "+pro.completiontime);
```

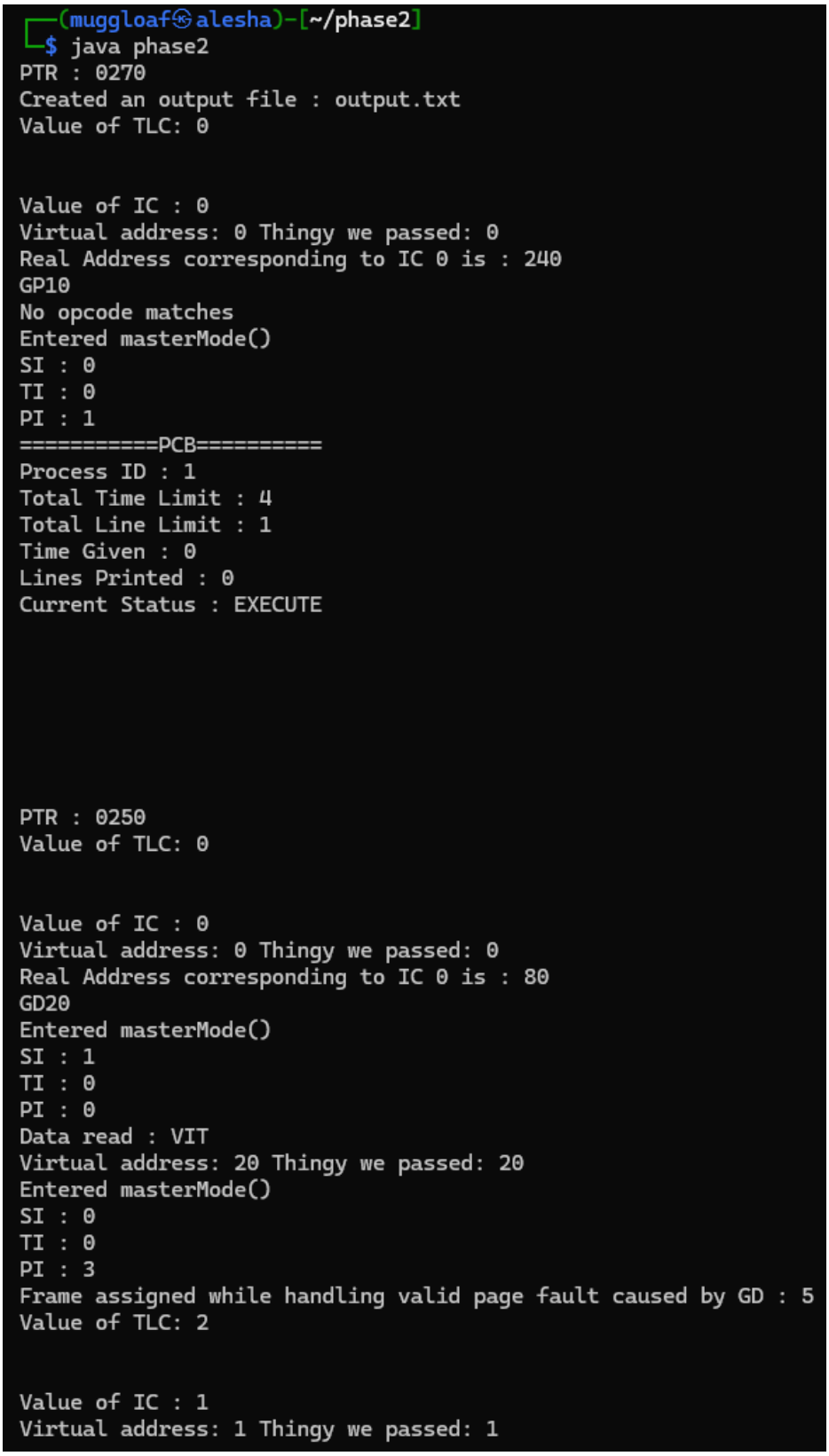
```
        System.out.println("Lines Printed : "+pro.linesprinted);
        System.out.println("Current Status : "+pro.currentstat);
        System.out.println("\n\n\n\n\n\n");
    }

     public static void terminate(String message, Scanner sc) throws
Exception{
        FileWriter fw=new FileWriter("output.txt",true);
        String IRcontent = "" + IR[0] + IR[1] + IR[2] + IR[3];
        String ICcontent = String.valueOf(IC);
          String temp="SI : "+SI+"   PI : "+PI+"   TI : "+TI+"    IR :
"+IRcontent+"   IC : "+ICcontent;


        if(OOD==1){
                        fw.write(message+"\n"+temp+"\n\n\n");
                        IC=numinstr;
                }
                else if(LLE==1 || PI!=0 || TI==2){
            fw.write(message+"\n"+temp+"\n\n\n");
            String card;
            String subcard;
            do{
                card=sc.nextLine();
                if(card.length()>4) subcard=card.substring(0,4);
                else subcard=card;
                IC = numinstr;
            }while(!subcard.equals("$END"));
        }
        else fw.write(message+"\n"+temp+"\n\n\n");
        fw.close();

        System.out.println("==========PCB==========");
        System.out.println("Process ID : "+pro.pid);
        System.out.println("Total Time Limit : "+pro.TTL);
        System.out.println("Total Line Limit : "+pro.TLL);
        System.out.println("Time Given : "+pro.completiontime);
        System.out.println("Lines Printed : "+pro.linesprinted);
        System.out.println("Current Status : "+pro.currentstat);
        System.out.println("\n\n\n\n\n\n");
    }
}
```

OUTPUT (SOME SAMPLE SCREENSHOTS)



```
┌──(muggloaf㉿alesha)-[~/phase2]
└─$ java phase2
PTR : 0270
Created an output file : output.txt
Value of TLC: 0


Value of IC : 0
Virtual address: 0 Thingy we passed: 0
Real Address corresponding to IC 0 is : 240
GP10
No opcode matches
Entered masterMode()
SI : 0
TI : 0
PI : 1
==========PCB==========
Process ID : 1
Total Time Limit : 4
Total Line Limit : 1
Time Given : 0
Lines Printed : 0
Current Status : EXECUTE









PTR : 0250
Value of TLC: 0


Value of IC : 0
Virtual address: 0 Thingy we passed: 0
Real Address corresponding to IC 0 is : 80
GD20
Entered masterMode()
SI : 1
TI : 0
PI : 0
Data read : VIT
Virtual address: 20 Thingy we passed: 20
Entered masterMode()
SI : 0
TI : 0
PI : 3
Frame assigned while handling valid page fault caused by GD : 5
Value of TLC: 2


Value of IC : 1
Virtual address: 1 Thingy we passed: 1
```

```
Real Address corresponding to IC 1 is : 81
GD30
Entered masterMode()
SI : 1
TI : 0
PI : 0
Data read : VIIT
Virtual address: 30 Thingy we passed: 30
Entered masterMode()
SI : 0
TI : 0
PI : 3
Frame assigned while handling valid page fault caused by GD : 7
Value of TLC: 4


Value of IC : 2
Virtual address: 2 Thingy we passed: 2
Real Address corresponding to IC 2 is : 82
GD40
Entered masterMode()
SI : 1
TI : 0
PI : 0
Data read : SAME
Virtual address: 40 Thingy we passed: 40
Entered masterMode()
SI : 0
TI : 0
PI : 3
Frame assigned while handling valid page fault caused by GD : 16
Value of TLC: 6


Value of IC : 3
Virtual address: 3 Thingy we passed: 3
Real Address corresponding to IC 3 is : 83
GD5O
Entered masterMode()
SI : 1
TI : 0
PI : 0
Data read : NOT SAME
Entered masterMode()
SI : 0
TI : 0
PI : 2
==========PCB==========
Process ID : 2
Total Time Limit : 15
Total Line Limit : 1
Time Given : 6
Lines Printed : 0
Current Status : EXECUTE
```

```
Value of IC : 10
Virtual address: 10 Thingy we passed: 10
Real Address corresponding to IC 10 is : 60
SR60
Virtual address: 60 Thingy we passed: 60
Entered masterMode()
SI : 0
TI : 0
PI : 3
SR was reset
Frame assigned while handling valid page fault caused by SR : 23
Value of TLC: 16


Value of IC : 11
Virtual address: 11 Thingy we passed: 11
Real Address corresponding to IC 11 is : 61
SR61
Virtual address: 61 Thingy we passed: 61
Value of TLC: 17


Value of IC : 12
Virtual address: 12 Thingy we passed: 12
Real Address corresponding to IC 12 is : 62
SR62
Virtual address: 62 Thingy we passed: 62
Value of TLC: 18


Value of IC : 13
Virtual address: 13 Thingy we passed: 13
Real Address corresponding to IC 13 is : 63
SR63
Virtual address: 63 Thingy we passed: 63
Value of TLC: 19


Value of IC : 14
Virtual address: 14 Thingy we passed: 14
Real Address corresponding to IC 14 is : 64
PD30
Entered masterMode()
SI : 2
TI : 0
PI : 0
Virtual address: 30 Thingy we passed: 30
Writing from block : 10 to output file
Value of TLC: 20


Value of IC : 15
Virtual address: 15 Thingy we passed: 15
Real Address corresponding to IC 15 is : 65
```

```
PD40
Entered masterMode()
SI : 2
TI : 0
PI : 0
==========PCB==========
Process ID : 3
Total Time Limit : 32
Total Line Limit : 1
Time Given : 20
Lines Printed : 2
Current Status : EXECUTE







PTR : 0240
Value of TLC: 0


Value of IC : 0
Virtual address: 0 Thingy we passed: 0
Real Address corresponding to IC 0 is : 50
GD30
Entered masterMode()
SI : 1
TI : 0
PI : 0
Data read : car
Virtual address: 30 Thingy we passed: 30
Entered masterMode()
SI : 0
TI : 0
PI : 3
Frame assigned while handling valid page fault caused by GD : 20
Value of TLC: 2


Value of IC : 1
Virtual address: 1 Thingy we passed: 1
Real Address corresponding to IC 1 is : 51
GD40
Entered masterMode()
SI : 1
TI : 2
PI : 0
==========PCB==========
Process ID : 4
Total Time Limit : 2
Total Line Limit : 1
Time Given : 2
Lines Printed : 2
```

```
Value of IC : 0
Virtual address: 0 Thingy we passed: 0
Real Address corresponding to IC 0 is : 160
GD20
Entered masterMode()
SI : 1
TI : 0
PI : 0
Data read : *
Virtual address: 20 Thingy we passed: 20
Entered masterMode()
SI : 0
TI : 0
PI : 3
Frame assigned while handling valid page fault caused by GD : 14
Value of TLC: 2


Value of IC : 1
Virtual address: 1 Thingy we passed: 1
Real Address corresponding to IC 1 is : 161
LR20
Virtual address: 20 Thingy we passed: 20
Value of TLC: 3


Value of IC : 2
Virtual address: 2 Thingy we passed: 2
Real Address corresponding to IC 2 is : 162
SR30
Virtual address: 30 Thingy we passed: 30
Entered masterMode()
SI : 0
TI : 0
PI : 3
SR was reset
Frame assigned while handling valid page fault caused by SR : 22
Value of TLC: 5


Value of IC : 3
Virtual address: 3 Thingy we passed: 3
Real Address corresponding to IC 3 is : 163
SR31
Virtual address: 31 Thingy we passed: 31
Value of TLC: 6


Value of IC : 4
Virtual address: 4 Thingy we passed: 4
Real Address corresponding to IC 4 is : 164
SR40
Virtual address: 40 Thingy we passed: 40
Entered masterMode()
```

```
Virtual address: 20 Thingy we passed: 20
Writing from block : 140 to output file
Value of TLC: 17


Value of IC : 14
Virtual address: 14 Thingy we passed: 14
Real Address corresponding to IC 14 is : 44
PD20
Entered masterMode()
SI : 2
TI : 0
PI : 0
Virtual address: 20 Thingy we passed: 20
Writing from block : 140 to output file
Value of TLC: 18


Value of IC : 15
Virtual address: 15 Thingy we passed: 15
Real Address corresponding to IC 15 is : 45
H---
Entered masterMode()
SI : 3
TI : 0
PI : 0
==========PCB==========
Process ID : 6
Total Time Limit : 22
Total Line Limit : 8
Time Given : 18
Lines Printed : 8
Current Status : EXECUTE









0       ----
1       ----
2       ----
3       ----
4       ----
5       ----
6       ----
7       ----
8       ----
9       ----
------------------------------------------------------------------
10      ----
11      ----
12      ----
13      ----
```

```
14      ----
15      ----
16      ----
17      ----
18      ----
19      ----
------------------------------------------------------------------
20      ----
21      ----
22      ----
23      ----
24      ----
25      ----
26      ----
27      ----
28      ----
29      ----
------------------------------------------------------------------
30      ----
31      ----
32      ----
33      ----
34      ----
35      ----
36      ----
37      ----
38      ----
39      ----
------------------------------------------------------------------
40      PD30
41      PD20
42      PD20
43      PD20
44      PD20
45      H---
46      ----
47      ----
48      ----
49      ----
------------------------------------------------------------------
50      ----
51      ----
52      ----
53      ----
54      ----
55      ----
56      ----
57      ----
58      ----
59      ----
------------------------------------------------------------------
60      ----
61      ----
62      ----
63      ----
```

CONCLUSION

To conclude, in phase 2 of implementing a multiprogramming operating system, we have introduced the concepts of virtual memory, paging, error handling, and PCBs. The virtual machine consisted of a 300 word physical memory, 100 word virtual memory, general purpose register, instruction register, instruction pointer, toggle bit, buffer register and page table register. 3 types of interrupts i.e. system interrupt, programming interrupt and timing interrupt were introduced. 7 types of errors were handled: operand error, opcode error, TLE, LLE, out of data and page faults. Pagining mechanism was successfully implemented. Thus, phase 2 is complete.